

电子商务管理系统

项目报告

学生姓名：王政南

学号：1030425120

班级：计科2501

提交日期：2026年7月1日

讲解视频网址：[https://www.bilibili.com/video/\[视频ID\]](https://www.bilibili.com/video/[视频ID])

目录

1. 项目概述
 2. 面向对象设计详解
 - 2.1 需求分析
 - 2.2 总体设计
 - 2.3 详细设计
 3. 核心难点与解决方案
 4. 编译与运行说明
 5. 总结与心得
 6. 附录：完整源代码
-

1. 项目概述

1.1 项目背景与目标

随着电子商务的普及，商品管理、购物车、订单处理等核心功能成为电商系统的基石。本项目旨在通过 C++ 面向对象编程技术，从零构建一个**控制台版电子商务管理系统**，模拟真实电商平台的核心业务流程。

项目目标：

- 深入实践面向对象三大特性：**封装、继承、多态**
- 掌握抽象基类与纯虚函数的设计方法
- 熟练运用 C++11 智能指针（`std::shared_ptr`）管理动态内存
- 实现完整的电商业务流程闭环：**用户管理 → 商品浏览 → 购物车 → 下单 → 订单管理**

1.2 主要功能

用户系统：注册、登录、管理员/顾客角色区分

商品管理（管理员）：

- 查看所有商品
- 添加商品（实体商品 / 数字商品）
- 删除商品、库存补货

购物系统（顾客）：

- 按分类浏览商品（实体/数字）
- 购物车增删改查
- 结算下单（自动扣减库存）

订单管理（管理员）：

- 查看所有订单、更新订单状态
- 取消订单（自动退款恢复库存）

1.3 运行环境与启动方式

项目	说明
操作系统	Windows 11 / Linux / macOS
编译器	g++ (MinGW-w64) 8.1.0 或更高
C++ 标准	C++11
编码	UTF-8 (Windows 控制台 <code>chcp 65001</code>)

编译命令：

```
g++ -std=c++11 -Wall -o ecommerce.exe main.cpp user.cpp product.cpp cart.cpp
order.cpp store.cpp
```

运行方式：直接双击 `ecommerce.exe` 或在终端中执行 `.\ecommerce.exe`。

1.4 运行截图

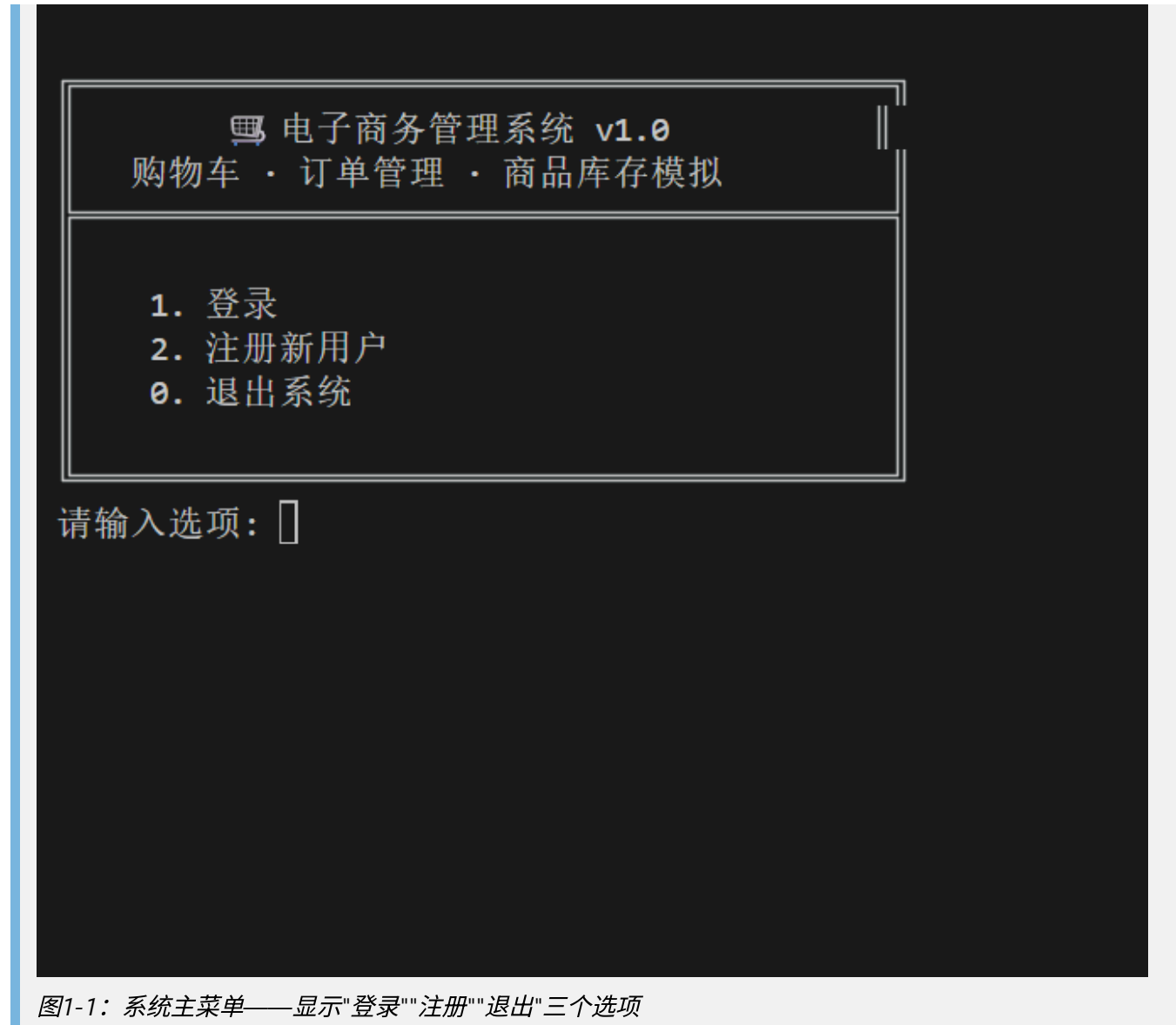


图1-1：系统主菜单——显示"登录""注册""退出"三个选项



图1-2：顾客菜单——浏览商品、添加到购物车、结算下单等操作界面

2. 面向对象设计详解

2.1 需求分析

2.1.1 项目描述

本系统是一个面向控制台的多角色电子商务管理平台。系统预设示例商品和用户数据，支持管理员对商品库存和订单状态进行后台管理，支持顾客浏览商品、操作购物车并完成下单。核心挑战在于：**商品类型多样**（实体/数字商品属性不同）、**用户角色不同**（管理员与顾客权限不同），需要通过合理继承结构统一管理，同时用多态简化业务逻辑。

2.1.2 功能需求

编号	功能	角色	说明
F1	用户注册与登录	所有用户	支持管理员/顾客两种角色注册

编号	功能	角色	说明
F2	商品浏览	所有用户	按全量或按分类（实体/数字）浏览商品
F3	商品添加	管理员	添加实体商品（含重量、配送区域）或数字商品（含文件大小、格式）
F4	商品删除	管理员	按商品ID删除
F5	库存补货	管理员	按商品ID增加库存数量
F6	加入购物车	顾客	验证库存后添加到购物车，已存在时合并数量
F7	购物车管理	顾客	查看、修改数量、移除商品
F8	结算下单	顾客	扣减库存，生成订单，清空购物车
F9	订单管理	管理员	查看所有订单、更新状态、取消订单（恢复库存）
F10	订单历史	顾客	查看个人历史订单

2.1.3 用例分析

用例名称：顾客完成一次购物

参与者：已登录的顾客

前置条件：顾客已成功登录，系统中存在有库存的商品

主事件流：

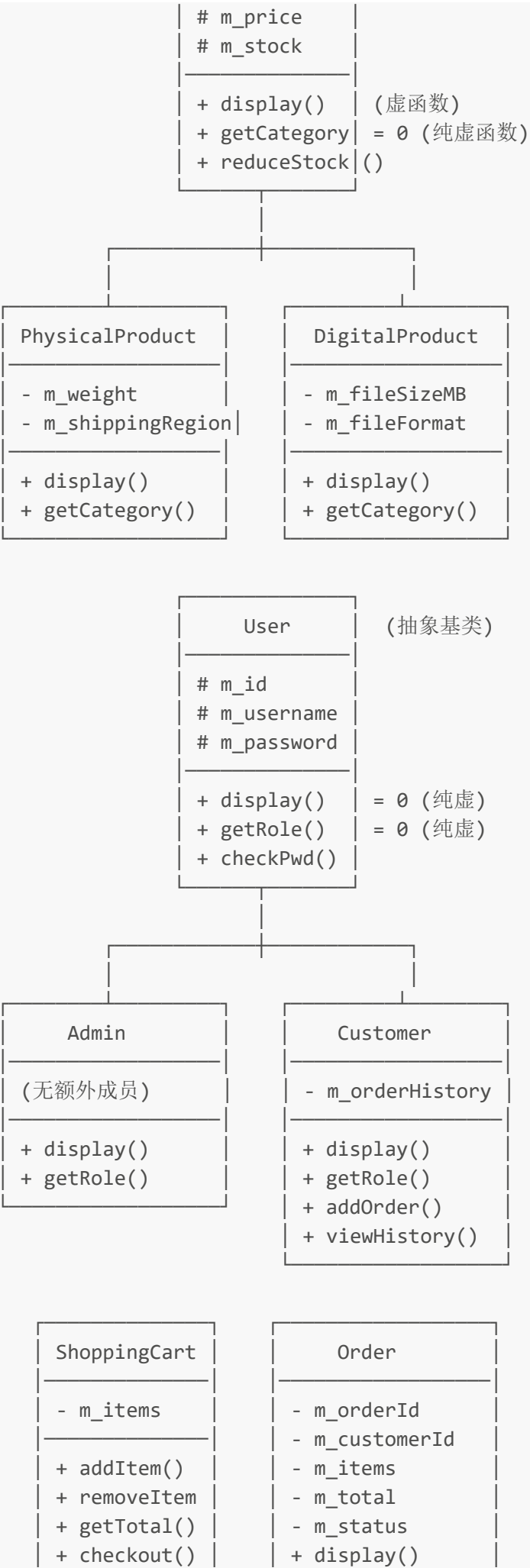
- 1. 顾客选择"浏览所有商品"，系统显示全部商品及库存
- 2. 顾客选择"添加商品到购物车"，输入商品ID和数量
- 3. 系统验证商品存在且库存充足，添加/更新购物车条目
- 4. 顾客选择"查看购物车"，确认商品和总价
- 5. 顾客选择"结算下单"，系统确认
- 6. 系统遍历购物车，逐一扣减对应商品库存
- 7. 系统生成订单（状态：待处理），清空购物车
- 8. 系统将订单记录到顾客的历史订单中

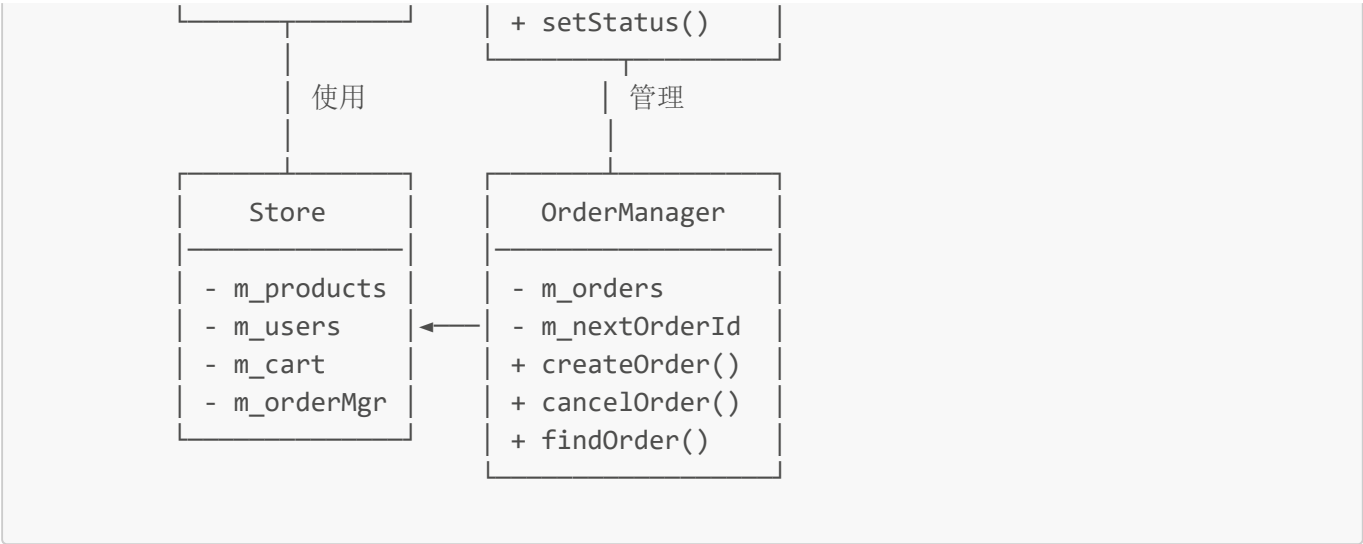
后置条件：订单创建成功，商品库存减少，购物车清空

2.2 总体设计

2.2.1 类继承图







- 图例说明：
- 空心三角箭头 (—>): 继承关系，箭头指向基类
 - 菱形箭头 (◁—): 组合/聚合关系
 - #: protected 成员; -: private 成员; +: public 成员
 - = 0: 纯虚函数

2.2.2 类职责说明

类名	类型	核心职责
Product	抽象基类	定义商品通用属性（id/名称/价格/库存），声明纯虚函数 getCategory() 作为分类接口，提供库存增减操作
PhysicalProduct	派生类	继承 Product，添加重量、配送区域属性，重写显示和分类函数
DigitalProduct	派生类	继承 Product，添加文件大小、文件格式属性，重写显示和分类函数
User	抽象基类	定义用户通用属性（id/用户名/密码），声明纯虚函数 display() 和 getRole()，提供密码校验
Admin	派生类	继承 User，实现管理员信息展示和角色标识
Customer	派生类	继承 User，额外维护订单历史列表，实现顾客特有功能（查看订单历史）
ShoppingCart	独立类	管理购物车商品列表，支持增删改查、总价计算、清空操作
Order	独立类	封装订单数据（商品明细/总价/状态/时间），支持状态流转和格式化展示
OrderManager	独立类	管理所有订单的创建、查询、状态更新和取消（含库存恢复）
Store	核心组合类	聚合所有子系统：用户管理、商品管理、订单管理，作为整个系统的门面

2.2.3 继承策略分析

为什么选择该继承结构？

1. **Product 继承体系**：实体商品和数字商品共享"名称、价格、库存"等基本属性，但各自有独特的附加属性（重量 vs 文件大小），且显示方式和分类标识不同。基类提取共性，派生类扩展个性，符合 **"开闭原则"**——未来新增"虚拟商品""服务类商品"只需新增派生类即可。
2. **User 继承体系**：管理员和顾客都拥有用户名、密码等基本认证信息，`checkPassword()` 逻辑完全一样，放在基类中**复用代码**；而角色标签和显示界面完全不同，通过纯虚函数 `getRole()` 和 `display()` 强制派生类各自实现，**实现多态接口**。

protected 成员设计理由：

- 基类 `Product` 的 `m_id`、`m_name`、`m_price`、`m_stock` 设为 `protected`：派生类 `PhysicalProduct::display()` 需调用基类 `Product::display()` 后再追加输出，派生类虽不直接访问这些成员（通过 getter），但保留 `protected` 是为了**未来可能的派生类直接操作基础属性需求**。
- 基类 `User` 的 `m_id`、`m_username`、`m_password` 设为 `protected`：`Admin::display()` 和 `Customer::display()` 直接引用 `m_id` 和 `m_username` 构造输出，这是 `protected` 的典型应用场景——**派生类访问基类成员，但不对外暴露**。

virtual 函数设计理由：

- `Product::display()` 声明为 `virtual` 但非纯虚：基类提供了默认的商品信息显示格式（ID/名称/价格/库存），派生类 `PhysicalProduct` 和 `DigitalProduct` 在基类显示基础上各自追加类型特有信息，体现了**多态复用**——基类指针调用 `display()` 时，运行时动态绑定到正确的派生类版本。
- `getCategory()` 和 `getRole()` 声明为**纯虚函数 = 0**：每个派生类必须返回不同的分类/角色标识文本，基类无法提供有意义的默认实现，纯虚函数强制派生类必须重写，同时使基类成为**抽象类**（不能直接实例化），这是正确的设计——不应存在"一个既不是实体也不是数字的 `Product` 对象"。

2.3 详细设计

2.3.1 关键类的接口设计

1. 抽象基类 `Product`

```
class Product {
protected:
    int         m_id;
    std::string m_name;
    double      m_price;
    int         m_stock;
    std::string m_description;

public:
    Product(int id, const std::string& name, double price, int stock,
            const std::string& description = "");
    virtual ~Product() = default;

    // 虚函数：基类提供默认实现，派生类可重写扩展
    virtual void display() const;
    // 纯虚函数：派生类必须实现，返回商品分类字符串
    virtual std::string getCategory() const = 0;

    // 公共接口（非虚函数，所有派生类行为一致）
```



```

int getId() const;
int getStock() const;
bool reduceStock(int quantity); // 扣减库存, true=成功
void increaseStock(int quantity); // 增加库存
};

```

2. 派生类 **PhysicalProduct**

```

class PhysicalProduct : public Product {
private:
    double m_weight; // 重量 (kg)
    std::string m_shippingRegion; // 配送区域

public:
    PhysicalProduct(int id, const std::string& name, double price, int stock,
                    double weight, const std::string& shippingRegion = "全国",
                    const std::string& description = "");

    void display() const override; // 重写: 先调基类display再追加
    std::string getCategory() const override; // 返回 "实体商品"
};

```

3. 核心组合类 **Store**

```

class Store {
private:
    std::vector<std::shared_ptr<Product>> m_products; // 多态容器
    std::vector<std::shared_ptr<User>> m_users; // 多态容器
    OrderManager m_orderManager;
    int m_nextProductId;
    int m_nextUserId;
    std::shared_ptr<User> m_currentUser;

public:
    // 用户管理
    std::shared_ptr<User> login(const std::string& username,
                               const std::string& password);
    std::shared_ptr<User> registerUser(const std::string& username,
                                       const std::string& password, bool isAdmin);

    // 商品管理
    void addProduct(std::shared_ptr<Product> product);
    bool removeProduct(int productId);

    // 购物与结算
    bool addToCart(ShoppingCart& cart, int productId, int quantity) const;
    std::shared_ptr<Order> checkout(ShoppingCart& cart, int customerId,
                                    const std::string& customerName);

    // 多态显示: 遍历基类指针调用虚函数
    void displayProducts() const;
};

```

```
void displayProductsByCategory(const std::string& category) const;
};
```

2.3.2 虚函数与多态的体现

以下是系统中多态的核心应用——通过基类指针容器统一管理派生类对象，调用虚函数实现动态绑定：

```
// ===== 场景1: 多态显示所有商品 =====
void Store::displayProducts() const {
    // m_products 是 vector<shared_ptr<Product>>
    // 实际存储的是 PhysicalProduct 或 DigitalProduct 的指针
    for (const auto& product : m_products) {
        product->display(); // ★ 多态调用!
        // 运行时根据实际对象类型分发:
        //   PhysicalProduct → PhysicalProduct::display()
        //   DigitalProduct  → DigitalProduct::display()
    }
}

// ===== 场景2: 按分类筛选商品 =====
void Store::displayProductsByCategory(const std::string& category) const {
    for (const auto& product : m_products) {
        if (product->getCategory() == category) { // ★ 多态调用纯虚函数
            product->display(); // ★ 多态调用虚函数
        }
    }
}

// ===== 场景3: 登录后根据角色分派菜单 =====
// 在 mainMenu() 中:
auto user = store.login(username, password);
if (user->getRole() == "管理员") { // ★ 基类指针调用纯虚函数
    adminMenu(store); // 管理员 → 管理界面
} else {
    customerMenu(store); // 顾客 → 购物界面
}
```

多态带来的架构优势：添加新商品类型时，只需新增一个 `Product` 的派生类并重写 `display()` 和 `getCategory()`，所有使用 `vector<shared_ptr<Product>>` 的代码（显示、筛选、购物车）**无需任何修改**，完美符合“对扩展开放，对修改关闭”的开闭原则。

2.3.3 核心算法/业务逻辑简介

结算下单流程（系统中最复杂的业务流程）：

```
用户确认下单
|
├─ 1. 校验购物车非空
```

- └ 2. 遍历购物车商品：
 - └ findProduct(id) → 获取存储中的真实商品对象
 - └ product->reduceStock(qty) → 扣减库存
 - └ 若任一商品库存不足 → 下单失败（返回 nullptr）
- └ 3. OrderManager.createOrder() → 创建订单（状态：Pending）
- └ 4. ShoppingCart.clear() → 清空购物车
- └ 5. 返回订单对象 → 调用方将其加入顾客历史

订单取消与库存恢复：

```
bool OrderManager::cancelOrder(int orderId) {
    auto order = findOrder(orderId);
    // 已签收或已取消的订单不可再取消
    if (order->getStatus() == OrderStatus::Delivered ||
        order->getStatus() == OrderStatus::Cancelled) {
        return false;
    }
    // 遍历订单商品，逐一恢复库存
    for (const auto& item : order->getItems()) {
        item.product->increaseStock(item.quantity);
    }
    order->setStatus(OrderStatus::Cancelled);
    return true;
}
```

3. 核心难点与解决方案

3.1 问题描述

在项目开发初期，我遇到了一个**悬垂指针（dangling pointer）**导致程序崩溃的问题。

问题场景：顾客将商品加入购物车时，我最初使用裸指针 `Product*` 存储商品的引用。当管理员在另一操作中删除该商品后，购物车中的指针变成了悬垂指针。之后顾客查看购物车或结算时，程序直接崩溃（访问已释放的内存）。

3.2 定位过程

1. **日志插桩：**在 `Store::removeProduct()` 和 `ShoppingCart::display()` 中添加 `std::cout` 输出，发现删除商品后再次访问购物车时崩溃发生在 `item.product->getName()` 调用处。

2. **GDB 调试：**

```
gdb ecommerce.exe
(gdb) run
```

```
# 触发崩溃后：
(gdb) bt          # 查看调用栈，定位崩溃点
(gdb) print item  # 查看指针地址，发现指向的内存已被释放
```

3. **根因确认**：多个对象持有同一 **Product** 的裸指针，缺乏统一的生命周期管理。当 Store 删除了商品，购物车和订单中的指针全部失效。

3.3 解决方案

将所有涉及对象的裸指针替换为 **C++11 智能指针 `std::shared_ptr`**，利用**引用计数**机制自动管理生命周期。

关键改动：

```
// ===== 改动前（有问题的设计） =====
class ShoppingCart {
    std::vector<std::pair<Product*, int>> m_items; // 裸指针！
};

class Store {
    std::vector<Product*> m_products; // 裸指针！
    // 删除商品时 delete，但购物车/订单中仍有引用 → 悬垂指针
};

// ===== 改动后（正确设计） =====
class ShoppingCart {
    // CartItem 持有 shared_ptr<Product>，引用计数+1
    std::vector<CartItem> m_items;
};

struct CartItem {
    std::shared_ptr<Product> product; // 智能指针！
    int quantity;
};

class Store {
    // 所有容器统一使用 shared_ptr
    std::vector<std::shared_ptr<Product>> m_products;
    std::vector<std::shared_ptr<User>> m_users;
};

// 删除商品时：从 m_products 中 erase → 引用计数-1
// 若购物车/订单中仍持有 shared_ptr → 引用计数 > 0 → 内存不被释放
// 当所有持有者都释放后 → 引用计数 = 0 → 内存自动释放
```

效果：彻底消除了悬垂指针问题。**`shared_ptr`** 的引用计数机制确保：只要还有任何一个对象（购物车、订单、商品列表）引用某商品，其内存就不会被释放。代码更加安全，且不需要手动 **`delete`**。

4. 编译与运行说明

4.1 环境信息

项目	信息
操作系统	Windows 11 Home China 10.0.26200
编译器	g++ (MinGW-w64)
编译器版本查询	<code>g++ --version</code>
C++ 标准	C++11

4.2 编译命令

逐个编译：

```
g++ -std=c++11 -Wall -g3 -c main.cpp -o main.o
g++ -std=c++11 -Wall -g3 -c user.cpp -o user.o
g++ -std=c++11 -Wall -g3 -c product.cpp -o product.o
g++ -std=c++11 -Wall -g3 -c cart.cpp -o cart.o
g++ -std=c++11 -Wall -g3 -c order.cpp -o order.o
g++ -std=c++11 -Wall -g3 -c store.cpp -o store.o
g++ -std=c++11 -Wall -g3 main.o user.o product.o cart.o order.o store.o -o
ecommerce.exe
```

一键编译：

```
g++ -std=c++11 -Wall -o ecommerce.exe main.cpp user.cpp product.cpp cart.cpp
order.cpp store.cpp
```

使用 Makefile：

```
make
```

4.3 运行示例

```
# Windows
.\ecommerce.exe
```

预期输出（登录后的顾客菜单）：

```
┌────────────────────────────────────────────────────────────────────────────────┐
│                                     电子商务管理系统 v1.0                                     │
│          购物车 · 订单管理 · 商品库存模拟          │
└────────────────────────────────────────────────────────────────────────────────┘
```

1. 登录

2. 注册新用户

0. 退出系统

请输入选项：1
用户名：user1
密 码：123456
✓ 登录成功！

顾客信息

ID：1002

用户名：user1

角色：顾客

历史订单：0 笔

顾客菜单

1. 浏览所有商品

2. 按分类浏览商品

3. 添加商品到购物车

4. 查看购物车

5. 修改购物车商品数量

6. 从购物车移除商品

7. 结算下单

8. 查看我的订单

0. 退出登录

4.4 预设账号

角色	用户名	密码
管理员	admin	admin123
顾客1	user1	123456
顾客2	user2	123456

5. 总结与心得

5.1 项目完成情况

已完成功能：

- ✓ 用户注册与登录（管理员/顾客双角色）
- ✓ 商品管理（添加/删除/补货，支持实体商品和数字商品）
- ✓ 购物车完整操作（增删改查、数量合并、总价计算）
- ✓ 结算下单（库存扣减、订单生成、购物车清空）
- ✓ 订单管理（状态流转：待处理→已确认→已发货→已签收）
- ✓ 订单取消与库存恢复

- ☒ 分类浏览、订单历史查询

未完成/可改进：

- ☐ 数据持久化（当前关闭程序后数据丢失，可引入文件读写）
- ☐ 结算回滚机制（库存扣减中途失败时，需回滚已扣减的库存）
- ☐ 密码加密存储（当前明文存储）

5.2 对面向对象设计的理解深化

通过本项目的完整开发过程，我对面向对象设计有了实质性理解：

1. **封装**：将数据和操作数据的函数绑定在一起，通过 `public` 接口控制访问权限。例如 `Product::reduceStock()` 内部做库存校验，外部无需关心实现细节。
2. **继承与代码复用**：基类 `Product` 中定义的属性（id、名称、价格）和方法（库存操作）被两个派生类直接复用，避免了重复代码。
3. **多态的实际价值**：`Store` 用 `vector<shared_ptr<Product>>` 一条语句管理两种商品类型，`display()` 一行调用即可正确分发到派生类版本。没有多态的话，需要用 `if-else` 或 `switch-case` 区分类型，代码将膨胀且难以扩展。
4. **抽象基类的约束力**：`getCategory() = 0` 纯虚函数强制每个派生类必须实现分类接口，从编译器层面防止“遗漏实现”的错误。

5.3 收获与不足

收获：

- 从零完成了 10 个类的设计、编码、调试全流程
- 掌握了 `shared_ptr` 的正确使用姿势，理解了 RAII 资源管理思想
- 学会了用 GDB 定位内存错误，积累了对 C++ 内存模型的直观认知

不足与改进方向：

- 当前系统缺少单元测试，未来应引入测试框架（如 Google Test）
- 项目未使用第三方库，代码完全自包含，适合初学阶段
- 可考虑引入设计模式（如观察者模式处理库存变更通知、策略模式处理折扣计算）

6. 附录：完整源代码

以下提供全部源代码。GitHub 仓库链接：[填写仓库地址]

6.1 product.h

```
#ifndef PRODUCT_H
#define PRODUCT_H

#include <string>
```

```
class Product {
protected:
    int          m_id;
    std::string m_name;
    double       m_price;
    int          m_stock;
    std::string m_description;

public:
    Product(int id, const std::string& name, double price, int stock,
            const std::string& description = "");
    virtual ~Product() = default;

    virtual void      display()      const;
    virtual std::string getCategory() const = 0;

    int          getId()          const { return m_id; }
    const std::string& getName()    const { return m_name; }
    double       getPrice()        const { return m_price; }
    int          getStock()         const { return m_stock; }
    const std::string& getDescription() const { return m_description; }

    void setId(int id)      { m_id = id; }
    void setStock(int stock) { m_stock = stock; }
    bool reduceStock(int quantity);
    void increaseStock(int quantity);
};

class PhysicalProduct : public Product {
private:
    double      m_weight;
    std::string m_shippingRegion;

public:
    PhysicalProduct(int id, const std::string& name, double price, int stock,
                    double weight, const std::string& shippingRegion = "全国",
                    const std::string& description = "");

    void      display()      const override;
    std::string getCategory() const override;

    double      getWeight()          const { return m_weight; }
    const std::string& getShippingRegion() const { return m_shippingRegion; }
};

class DigitalProduct : public Product {
private:
    double      m_fileSizeMB;
    std::string m_fileFormat;

public:
    DigitalProduct(int id, const std::string& name, double price, int stock,
                    double fileSizeMB, const std::string& fileFormat = "通用",
                    const std::string& description = "");
};
```



```

void      display()      const override;
std::string getCategory() const override;

double      getFileSize() const { return m_fileSizeMB; }
const std::string& getFileFormat() const { return m_fileFormat; }
};

#endif // PRODUCT_H

```

6.2 product.cpp

```

#include "product.h"
#include <iostream>
#include <iomanip>

Product::Product(int id, const std::string& name, double price, int stock,
                 const std::string& description)
: m_id(id), m_name(name), m_price(price), m_stock(stock),
  m_description(description) {}

void Product::display() const {
    std::cout << "[" << m_id << "]" " << m_name
               << " | ¥" << std::fixed << std::setprecision(2) << m_price
               << " | 库存: " << m_stock;
    if (!m_description.empty()) {
        std::cout << " | " << m_description;
    }
    std::cout << "\n";
}

bool Product::reduceStock(int quantity) {
    if (quantity <= 0 || m_stock < quantity) {
        return false;
    }
    m_stock -= quantity;
    return true;
}

void Product::increaseStock(int quantity) {
    if (quantity > 0) {
        m_stock += quantity;
    }
}

PhysicalProduct::PhysicalProduct(int id, const std::string& name, double price,
                                  int stock, double weight,
                                  const std::string& shippingRegion,
                                  const std::string& description)
: Product(id, name, price, stock, description),
  m_weight(weight), m_shippingRegion(shippingRegion) {}

```

```

void PhysicalProduct::display() const {
    Product::display();
    std::cout << "    [实体商品] 重量: " << m_weight << "kg"
                << ", 配送区域: " << m_shippingRegion << "\n";
}

std::string PhysicalProduct::getCategory() const {
    return "实体商品";
}

DigitalProduct::DigitalProduct(int id, const std::string& name, double price,
                                int stock, double fileSizeMB,
                                const std::string& fileFormat,
                                const std::string& description)
    : Product(id, name, price, stock, description),
      m_fileSizeMB(fileSizeMB), m_fileFormat(fileFormat) {}

void DigitalProduct::display() const {
    Product::display();
    std::cout << "    [数字商品] 大小: " << m_fileSizeMB << "MB"
                << ", 格式: " << m_fileFormat << "\n";
}

std::string DigitalProduct::getCategory() const {
    return "数字商品";
}

```

6.3 user.h

```

#ifndef USER_H
#define USER_H

#include <string>
#include <vector>
#include <memory>

class Order;

class User {
protected:
    int m_id;
    std::string m_username;
    std::string m_password;

public:
    User(int id, const std::string& username, const std::string& password);
    virtual ~User() = default;

    virtual void display() const = 0;
    virtual std::string getRole() const = 0;
}

```

```

    int            getId()      const { return m_id; }
    const std::string& getUsername() const { return m_username; }

    bool checkPassword(const std::string& pwd) const { return m_password == pwd; }
};

class Admin : public User {
public:
    Admin(int id, const std::string& username, const std::string& password);

    void display() const override;
    std::string getRole() const override;
};

class Customer : public User {
private:
    std::vector<std::shared_ptr<Order>> m_orderHistory;

public:
    Customer(int id, const std::string& username, const std::string& password);

    void display() const override;
    std::string getRole() const override;

    void addOrder(std::shared_ptr<Order> order);
    void viewOrderHistory() const;
    const std::vector<std::shared_ptr<Order>>& getOrders() const;
};

#endif // USER_H

```

6.4 user.cpp

```

#include "user.h"
#include "order.h"
#include <iostream>

User::User(int id, const std::string& username, const std::string& password)
    : m_id(id), m_username(username), m_password(password) {}

Admin::Admin(int id, const std::string& username, const std::string& password)
    : User(id, username, password) {}

void Admin::display() const {
    std::cout << "      管理员信息      \n";
    std::cout << "  ID:      " << m_id << "\n";
    std::cout << "  用户名: " << m_username << "\n";
    std::cout << "  角色:    " << getRole() << "\n";
    std::cout << "      \n";
}

```

```

std::string Admin::getRole() const {
    return "管理员";
}

Customer::Customer(int id, const std::string& username, const std::string&
password)
    : User(id, username, password) {}

void Customer::display() const {
    std::cout << "      顾客信息      \n";
    std::cout << "  ID:      " << m_id << "\n";
    std::cout << "  用户名: " << m_username << "\n";
    std::cout << "  角色:   " << getRole() << "\n";
    std::cout << "  历史订单: " << m_orderHistory.size() << "  笔\n";
    std::cout << "      \n";
}

std::string Customer::getRole() const {
    return "顾客";
}

void Customer::addOrder(std::shared_ptr<Order> order) {
    m_orderHistory.push_back(std::move(order));
}

void Customer::viewOrderHistory() const {
    if (m_orderHistory.empty()) {
        std::cout << "  暂无订单记录。 \n";
        return;
    }
    for (const auto& order : m_orderHistory) {
        order->display();
    }
}

const std::vector<std::shared_ptr<Order>>& Customer::getOrders() const {
    return m_orderHistory;
}

```

6.5 cart.h

```

#ifndef CART_H
#define CART_H

#include <vector>
#include <memory>

class Product;

struct CartItem {

```

```

        std::shared_ptr<Product> product;
        int quantity;

        CartItem(std::shared_ptr<Product> p, int qty)
            : product(std::move(p)), quantity(qty) {}
    };

    class ShoppingCart {
    private:
        std::vector<CartItem> m_items;

    public:
        ShoppingCart() = default;

        void addItem(std::shared_ptr<Product> product, int quantity);
        void removeItem(int productId);
        void updateQuantity(int productId, int newQuantity);
        void clear();

        double getTotal() const;
        int getItemCount() const { return static_cast<int>
(m_items.size()); }
        bool isEmpty() const { return m_items.empty(); }
        const std::vector<CartItem>& getItems() const { return m_items; }

        void display() const;
    };

#endif // CART_H

```

6.6 cart.cpp

```

#include "cart.h"
#include "product.h"
#include <iostream>
#include <iomanip>
#include <algorithm>

void ShoppingCart::addItem(std::shared_ptr<Product> product, int quantity) {
    if (quantity <= 0) return;

    for (auto& item : m_items) {
        if (item.product->getId() == product->getId()) {
            item.quantity += quantity;
            std::cout << " 已更新 \"" << product->getName()
                << "\" 数量为 " << item.quantity << "\n";
            return;
        }
    }
    m_items.emplace_back(std::move(product), quantity);
    std::cout << " 已添加 \"" << m_items.back().product->getName()

```

```

        << "\"" << x << " 到购物车\n";
    }

    void ShoppingCart::removeItem(int productId) {
        auto it = std::find_if(m_items.begin(), m_items.end(),
            [productId](const CartItem& item) {
                return item.product->getId() == productId;
            });
        if (it != m_items.end()) {
            std::cout << " 已从购物车移除 \"" << it->product->getName() << "\"\n";
            m_items.erase(it);
        } else {
            std::cout << " 购物车中未找到该商品。 \n";
        }
    }

    void ShoppingCart::updateQuantity(int productId, int newQuantity) {
        if (newQuantity <= 0) {
            removeItem(productId);
            return;
        }
        for (auto& item : m_items) {
            if (item.product->getId() == productId) {
                item.quantity = newQuantity;
                std::cout << " 已更新 \"" << item.product->getName()
                    << "\" 数量为 " << newQuantity << "\n";
                return;
            }
        }
        std::cout << " 购物车中未找到该商品。 \n";
    }

    void ShoppingCart::clear() {
        m_items.clear();
    }

    double ShoppingCart::getTotal() const {
        double total = 0.0;
        for (const auto& item : m_items) {
            total += item.product->getPrice() * item.quantity;
        }
        return total;
    }

    void ShoppingCart::display() const {
        if (m_items.empty()) {
            std::cout << " 购物车为空。 \n";
            return;
        }
        std::cout << "\n|----- 购物车 -----|\n";
        std::cout << "| 序号 商品名称          单价      数量  小计\n";
        std::cout << "|-----| \n";
        int index = 1;
        for (const auto& item : m_items) {

```

```

        double subtotal = item.product->getPrice() * item.quantity;
        std::cout << " | " << std::setw(2) << index++ << ". "
            << std::setw(20) << std::left << item.product->getName()
            << std::right
            << " ¥" << std::setw(7) << std::fixed << std::setprecision(2) <<
item.product->getPrice()
            << " x" << std::setw(3) << item.quantity
            << " ¥" << std::setw(8) << subtotal << "\n";
    }
    std::cout << " | _____ |\n";
    std::cout << " | 合计:                ¥"
        << std::setw(8) << getTotal() << "\n";
    std::cout << " | _____ |\n";
}

```

6.7 order.h

```

#ifndef ORDER_H
#define ORDER_H

#include <string>
#include <vector>
#include <memory>
#include "cart.h"

enum class OrderStatus {
    Pending,
    Confirmed,
    Shipped,
    Delivered,
    Cancelled
};

class Order {
private:
    int m_orderId;
    int m_customerId;
    std::string m_customerName;
    std::vector<CartItem> m_items;
    double m_total;
    OrderStatus m_status;
    std::string m_orderTime;

public:
    Order(int orderId, int customerId, const std::string& customerName,
        const std::vector<CartItem>& items);

    int getOrderId() const { return m_orderId; }
    int getCustomerId() const { return m_customerId; }
    const std::string& getCustomerName() const { return m_customerName; }
    double getTotal() const { return m_total; }

```

```

    OrderStatus      getStatus()      const { return m_status; }
    const std::string& getOrderTime()  const { return m_orderTime; }
    const std::vector<CartItem>& getItems() const { return m_items; }

    void setStatus(OrderStatus status);
    std::string getStatusString() const;
    static std::string statusToString(OrderStatus s);

    void display() const;
};

class OrderManager {
private:
    std::vector<std::shared_ptr<Order>> m_orders;
    int m_nextOrderId;

public:
    OrderManager();

    std::shared_ptr<Order> createOrder(int customerId, const std::string&
customerName,
                                   const std::vector<CartItem>& items);

    bool cancelOrder(int orderId);
    bool updateOrderStatus(int orderId, OrderStatus newStatus);

    std::shared_ptr<Order> findOrder(int orderId) const;
    std::vector<std::shared_ptr<Order>> getOrdersByCustomer(int customerId) const;
    const std::vector<std::shared_ptr<Order>>& getAllOrders() const { return
m_orders; }
    int getOrderCount() const { return static_cast<int>(m_orders.size()); }

    void displayAllOrders() const;
    void displayOrdersByCustomer(int customerId) const;
};

#endif // ORDER_H

```

6.8 order.cpp

```

#include "order.h"
#include "product.h"
#include <iostream>
#include <iomanip>
#include <ctime>
#include <sstream>
#include <algorithm>

Order::Order(int orderId, int customerId, const std::string& customerName,
             const std::vector<CartItem>& items)
    : m_orderId(orderId), m_customerId(customerId), m_customerName(customerName),
      m_items(items), m_status(OrderStatus::Pending)

```



```

{
    m_total = 0.0;
    for (const auto& item : m_items) {
        m_total += item.product->getPrice() * item.quantity;
    }

    std::time_t now = std::time(nullptr);
    std::tm* localTime = std::localtime(&now);
    std::ostringstream oss;
    oss << (localTime->tm_year + 1900) << "-"
        << std::setw(2) << std::setfill('0') << (localTime->tm_mon + 1) << "-"
        << std::setw(2) << std::setfill('0') << localTime->tm_mday << " "
        << std::setw(2) << std::setfill('0') << localTime->tm_hour << ":"
        << std::setw(2) << std::setfill('0') << localTime->tm_min << ":"
        << std::setw(2) << std::setfill('0') << localTime->tm_sec;
    m_orderTime = oss.str();
}

void Order::setStatus(OrderStatus status) {
    m_status = status;
}

std::string Order::getStatusString() const {
    return statusToString(m_status);
}

std::string Order::statusToString(OrderStatus s) {
    switch (s) {
        case OrderStatus::Pending:    return "待处理";
        case OrderStatus::Confirmed:  return "已确认";
        case OrderStatus::Shipped:    return "已发货";
        case OrderStatus::Delivered:  return "已签收";
        case OrderStatus::Cancelled:  return "已取消";
        default:                      return "未知";
    }
}

void Order::display() const {
    std::cout << "\n" << "----- 订单详情 -----" << "\n";
    std::cout << " | 订单编号: #" << m_orderId << "\n";
    std::cout << " | 顾客:      " << m_customerName << " (ID:" << m_customerId <<
    ") \n";
    std::cout << " | 下单时间: " << m_orderTime << "\n";
    std::cout << " | 状态:      " << getStatusString() << "\n";
    std::cout << " |-----" << "\n";
    for (const auto& item : m_items) {
        double subtotal = item.product->getPrice() * item.quantity;
        std::cout << " | . " << std::setw(22) << std::left << item.product-
        >getName()
            << std::right << " x" << item.quantity
            << " ¥" << std::fixed << std::setprecision(2) << subtotal <<
        "\n";
    }
    std::cout << " |-----" << "\n";
}

```

```
std::cout << "| 合计: ¥" << std::fixed << std::setprecision(2) << m_total <<
"\n";
std::cout << "└──────────────────────────────────┘\n";
}

OrderManager::OrderManager() : m_nextOrderId(10001) {}

std::shared_ptr<Order> OrderManager::createOrder(
    int customerId, const std::string& customerName,
    const std::vector<CartItem>& items)
{
    auto order = std::make_shared<Order>(m_nextOrderId++, customerName, items);
    m_orders.push_back(order);
    return order;
}

bool OrderManager::cancelOrder(int orderId) {
    auto order = findOrder(orderId);
    if (!order) return false;
    if (order->getStatus() == OrderStatus::Delivered ||
        order->getStatus() == OrderStatus::Cancelled) {
        return false;
    }
    for (const auto& item : order->getItems()) {
        item.product->increaseStock(item.quantity);
    }
    order->setStatus(OrderStatus::Cancelled);
    return true;
}

bool OrderManager::updateOrderStatus(int orderId, OrderStatus newStatus) {
    auto order = findOrder(orderId);
    if (!order) return false;
    order->setStatus(newStatus);
    return true;
}

std::shared_ptr<Order> OrderManager::findOrder(int orderId) const {
    auto it = std::find_if(m_orders.begin(), m_orders.end(),
        [orderId](const std::shared_ptr<Order>& o) {
            return o->getOrderId() == orderId;
        });
    return (it != m_orders.end()) ? *it : nullptr;
}

std::vector<std::shared_ptr<Order>> OrderManager::getOrdersByCustomer(
    int customerId) const
{
    std::vector<std::shared_ptr<Order>> result;
    for (const auto& order : m_orders) {
        if (order->getCustomerId() == customerId) {
            result.push_back(order);
        }
    }
}
```

```

    }
    return result;
}

void OrderManager::displayAllOrders() const {
    if (m_orders.empty()) {
        std::cout << "  暂无订单。\\n";
        return;
    }
    std::cout << "\\n|_____ 所有订单 _____|\\n";
    for (const auto& order : m_orders) {
        std::cout << "| #" << order->getOrderId()
                  << " | " << order->getCustomerName()
                  << " | ¥" << std::fixed << std::setprecision(2) << order-
>getTotal()
                  << " | " << order->getStatusString() << "\\n";
    }
    std::cout << "|_____|\\n";
}

void OrderManager::displayOrdersByCustomer(int customerId) const {
    auto orders = getOrdersByCustomer(customerId);
    if (orders.empty()) {
        std::cout << "  该顾客暂无订单。\\n";
        return;
    }
    for (const auto& order : orders) {
        order->display();
    }
}

```

6.9 store.h

```

#ifndef STORE_H
#define STORE_H

#include <vector>
#include <memory>
#include <string>
#include "product.h"
#include "user.h"
#include "order.h"
#include "cart.h"

class Store {
private:
    std::vector<std::shared_ptr<Product>> m_products;
    std::vector<std::shared_ptr<User>> m_users;
    OrderManager m_orderManager;
    int m_nextProductId;
    int m_nextUserId;

```

```

        std::shared_ptr<User> m_currentUser;

public:
    Store();

    void initSampleData();

    std::shared_ptr<User> login(const std::string& username, const std::string&
password);
    std::shared_ptr<User> registerUser(const std::string& username,
                                     const std::string& password, bool isAdmin);
    void logout();
    std::shared_ptr<User> getCurrentUser() const { return m_currentUser; }
    const std::vector<std::shared_ptr<User>>& getAllUsers() const { return
m_users; }

    void addProduct(std::shared_ptr<Product> product);
    bool removeProduct(int productId);
    bool restockProduct(int productId, int quantity);
    std::shared_ptr<Product> findProduct(int productId) const;
    const std::vector<std::shared_ptr<Product>>& getAllProducts() const { return
m_products; }

    bool addToCart(ShoppingCart& cart, int productId, int quantity) const;
    std::shared_ptr<Order> checkout(ShoppingCart& cart, int customerId,
                                   const std::string& customerName);

    OrderManager& getOrderManager() { return m_orderManager; }

    void displayProducts() const;
    void displayProductsByCategory(const std::string& category) const;
};

#endif // STORE_H

```

6.10 main.cpp

```

/**
 * 电子商务系统 — 购物车、订单管理、商品库存模拟
 *
 * 面向对象设计:
 *   User  ─── Admin    (管理员: 管理商品、处理订单)
 *           └── Customer (顾客: 浏览商品、购物车、下单)
 *   Product ─── PhysicalProduct (实体商品: 含重量、配送信息)
 *               └── DigitalProduct (数字商品: 含文件大小、格式)
 *
 * 组合类: ShoppingCart, Order, OrderManager, Store
 *
 * 编译方式 (g++):
 *   g++ -std=c++11 -Wall -o ecommerce.exe main.cpp user.cpp product.cpp cart.cpp
order.cpp store.cpp

```

```
*/

#include <iostream>
#include <string>
#include <limits>
#include <memory>
#include "store.h"

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void pauseScreen() {
    std::cout << "\n按 Enter 键继续...";
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

void printTitle(const std::string& title) {
    std::cout << "\n";
    std::cout << "┌───────────────────────────────────────────────────────────────────────────────────┐\n";
    std::cout << "│          " << title;
    for (size_t i = title.size(); i < 40; i += 2) {
        std::cout << "  ";
    }
    std::cout << "\n";
    std::cout << "└───────────────────────────────────────────────────────────────────────────────────┘\n";
}

int readInt(const std::string& prompt = "请输入选项：") {
    int value;
    while (true) {
        std::cout << prompt;
        std::cin >> value;
        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << " ⚠ 请输入有效数字。 \n";
        } else {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return value;
        }
    }
}

double readDouble(const std::string& prompt) {
    double value;
    while (true) {
        std::cout << prompt;
        std::cin >> value;
        if (std::cin.fail() || value < 0) {
```

```

        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "    ⚠ 请输入有效数字。\\n";
    } else {
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return value;
    }
}

std::string readString(const std::string& prompt) {
    std::string value;
    std::cout << prompt;
    std::getline(std::cin, value);
    return value;
}

void adminMenu(Store& store) {
    while (true) {
        clearScreen();
        auto user = store.getCurrentUser();
        user->display();

        std::cout << "\\n|----- 管理员菜单 -----|\\n";
        std::cout << "| 1. 查看所有商品 |\\n";
        std::cout << "| 2. 添加商品（实体/数字） |\\n";
        std::cout << "| 3. 删除商品 |\\n";
        std::cout << "| 4. 补货商品 |\\n";
        std::cout << "| 5. 查看所有订单 |\\n";
        std::cout << "| 6. 更新订单状态 |\\n";
        std::cout << "| 7. 取消订单（退款+恢复库存） |\\n";
        std::cout << "| 0. 退出登录 |\\n";
        std::cout << "|-----|\\n";

        int choice = readInt();

        switch (choice) {
            case 1: {
                clearScreen();
                printTitle("商品列表");
                store.displayProducts();
                pauseScreen();
                break;
            }
            case 2: {
                clearScreen();
                printTitle("添加商品");
                std::cout << "选择商品类型:\\n";
                std::cout << "    1. 实体商品\\n";
                std::cout << "    2. 数字商品\\n";
                int type = readInt("请选择: ");

                std::string name = readString("商品名称: ");
                double price = readDouble("价格 (¥): ");
            }
        }
    }
}

```

```
int stock = readInt("初始库存: ");
std::string desc = readString("描述 (可选): ");

if (type == 1) {
    double weight = readDouble("重量 (kg): ");
    std::string region = readString("配送区域: ");
    auto p = std::make_shared<PhysicalProduct>(
        0, name, price, stock, weight, region, desc);
    store.addProduct(p);
} else {
    double size = readDouble("文件大小 (MB): ");
    std::string format = readString("文件格式: ");
    auto p = std::make_shared<DigitalProduct>(
        0, name, price, stock, size, format, desc);
    store.addProduct(p);
}
pauseScreen();
break;
}

case 3: {
    clearScreen();
    printTitle("删除商品");
    store.displayProducts();
    int id = readInt("输入要删除的商品ID: ");
    store.removeProduct(id);
    pauseScreen();
    break;
}

case 4: {
    clearScreen();
    printTitle("补货商品");
    store.displayProducts();
    int id = readInt("输入商品ID: ");
    int qty = readInt("补货数量: ");
    store.restockProduct(id, qty);
    pauseScreen();
    break;
}

case 5: {
    clearScreen();
    printTitle("所有订单");
    store.getOrderManager().displayAllOrders();
    pauseScreen();
    break;
}

case 6: {
    clearScreen();
    printTitle("更新订单状态");
    store.getOrderManager().displayAllOrders();
    int orderId = readInt("输入订单编号: ");
    auto order = store.getOrderManager().findOrder(orderId);
    if (!order) {
        std::cout << "    X  订单不存在。\\n";
    } else {
```

```

        std::cout << "当前状态: " << order->getStatusString() << "\n";
        std::cout << "可选状态:\n";
        std::cout << "  1. 已确认  2. 已发货  3. 已签收  4. 已取消\n";
        int s = readInt("选择新状态: ");
        OrderStatus statuses[] = {OrderStatus::Confirmed,
OrderStatus::Shipped,
                                OrderStatus::Delivered,
OrderStatus::Cancelled};
        if (s >= 1 && s <= 4) {
            store.getOrderManager().updateOrderStatus(orderId, statuses[s
- 1]);

            std::cout << "  ✓ 状态已更新。 \n";
        } else {
            std::cout << "  ✗ 无效选择。 \n";
        }
    }
    pauseScreen();
    break;
}
case 7: {
    clearScreen();
    printTitle("取消订单");
    store.getOrderManager().displayAllOrders();
    int orderId = readInt("输入要取消的订单编号: ");
    if (store.getOrderManager().cancelOrder(orderId)) {
        std::cout << "  ✓ 订单已取消, 库存已恢复。 \n";
    } else {
        std::cout << "  ✗ 取消失败 (订单不存在或不可取消)。 \n";
    }
    pauseScreen();
    break;
}
case 0:
    store.logout();
    return;
default:
    std::cout << "  ⚠ 无效选项, 请重新选择。 \n";
    pauseScreen();
}
}
}

void customerMenu(Store& store) {
    ShoppingCart cart;
    auto customer = std::dynamic_pointer_cast<Customer>(store.getCurrentUser());

    while (true) {
        clearScreen();
        auto user = store.getCurrentUser();
        user->display();

        std::cout << "\n┌────────────────── 顾客菜单 ───────────────────┐ \n";
        std::cout << "│ 1. 浏览所有商品 │ \n";
        std::cout << "│ 2. 按分类浏览商品 │ \n";
    }
}

```



```
std::cout << " | 3. 添加商品到购物车 | \n";
std::cout << " | 4. 查看购物车 | \n";
std::cout << " | 5. 修改购物车商品数量 | \n";
std::cout << " | 6. 从购物车移除商品 | \n";
std::cout << " | 7. 结算下单 | \n";
std::cout << " | 8. 查看我的订单 | \n";
std::cout << " | 0. 退出登录 | \n";
std::cout << " | \n";

int choice = readInt();

switch (choice) {
case 1: {
    clearScreen();
    printTitle("所有商品");
    store.displayProducts();
    pauseScreen();
    break;
}
case 2: {
    clearScreen();
    printTitle("按分类浏览");
    std::cout << " 1. 实体商品\n";
    std::cout << " 2. 数字商品\n";
    int c = readInt("选择分类: ");
    if (c == 1) {
        store.displayProductsByCategory("实体商品");
    } else if (c == 2) {
        store.displayProductsByCategory("数字商品");
    } else {
        std::cout << " ⚠ 无效选择。 \n";
    }
    pauseScreen();
    break;
}
case 3: {
    clearScreen();
    printTitle("添加到购物车");
    store.displayProducts();
    int id = readInt("输入商品ID: ");
    int qty = readInt("数量: ");
    store.addToCart(cart, id, qty);
    pauseScreen();
    break;
}
case 4: {
    clearScreen();
    printTitle("我的购物车");
    cart.display();
    pauseScreen();
    break;
}
case 5: {
    clearScreen();
```

```
    printTitle("修改商品数量");
    cart.display();
    if (!cart.isEmpty()) {
        int id = readInt("输入商品ID: ");
        int qty = readInt("新数量: ");
        cart.updateQuantity(id, qty);
    }
    pauseScreen();
    break;
}
case 6: {
    clearScreen();
    printTitle("移除商品");
    cart.display();
    if (!cart.isEmpty()) {
        int id = readInt("输入要移除的商品ID: ");
        cart.removeItem(id);
    }
    pauseScreen();
    break;
}
case 7: {
    clearScreen();
    printTitle("结算下单");
    cart.display();
    if (!cart.isEmpty()) {
        std::cout << "确认下单? (y/n): ";
        std::string confirm;
        std::getline(std::cin, confirm);
        if (confirm == "y" || confirm == "Y") {
            auto order = store.checkout(cart, customer->getId(),
                                       customer->getUsername());

            if (order) {
                customer->addOrder(order);
            }
        } else {
            std::cout << " 已取消。\\n";
        }
    }
    pauseScreen();
    break;
}
case 8: {
    clearScreen();
    printTitle("我的订单");
    customer->viewOrderHistory();
    pauseScreen();
    break;
}
case 0:
    store.logout();
    return;
default:
    std::cout << " ⚠ 无效选项, 请重新选择。\\n";
```

```

        pauseScreen();
    }
}

void mainMenu(Store& store) {
    while (true) {
        clearScreen();
        std::cout << "\n";
        std::cout << "
        |           🛒 电子商务管理系统 v1.0           | \n";
        std::cout << "        购物车 · 订单管理 · 商品库存模拟        | \n";
        std::cout << "
        |_____| \n";
        std::cout << "
        | \n";
        std::cout << "        1. 登录 \n";
        std::cout << "        2. 注册新用户 \n";
        std::cout << "        0. 退出系统 \n";
        std::cout << "
        |_____| \n";
        std::cout << "
        |_____| \n";

        int choice = readInt();

        switch (choice) {
        case 1: {
            clearScreen();
            printTitle("用户登录");
            std::string username = readString("用户名: ");
            std::string password = readString("密 码: ");

            auto user = store.login(username, password);
            if (user) {
                std::cout << "    ✓ 登录成功! \n";
                pauseScreen();
                if (user->getRole() == "管理员") {
                    adminMenu(store);
                } else {
                    customerMenu(store);
                }
            } else {
                std::cout << "    ✗ 用户名或密码错误. \n";
                pauseScreen();
            }
            break;
        }
        case 2: {
            clearScreen();
            printTitle("注册新用户");
            std::string username = readString("用户名: ");
            std::string password = readString("密 码: ");
            std::cout << "注册类型 (1=顾客, 2=管理员): ";
            int type;
            std::cin >> type;
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

```

```
        auto newUser = store.registerUser(username, password, type == 2);
        if (newUser) {
            pauseScreen();
            if (newUser->getRole() == "管理员") {
                adminMenu(store);
            } else {
                customerMenu(store);
            }
        } else {
            pauseScreen();
        }
        break;
    }
    case 0:
        std::cout << "\n 感谢使用! 再见! \n";
        return;
    default:
        std::cout << " ⚠ 无效选项。 \n";
        pauseScreen();
    }
}

}

int main() {
#ifdef _WIN32
    system("chcp 65001 > nul");
#endif

    Store store;
    store.initSampleData();

    mainMenu(store);
    return 0;
}
```